PYTHON ACTIVITY-BASED LEARNING ASSIGNMENT

Documentation for 10 Selected Case Studies

Student Name: N Mary Jyothsna

Roll Number: 26001405

Branch / Section: CSD-1

Course: Python Programming

Academic Year: 2026–2027

Note: The output examples are expected results. Run each program on your laptop and insert your own output screenshots before submission.

CASE STUDY 1: Attendance Recovery Computation

1. Problem Statement

Write a Python program that calculates a student's current attendance percentage and estimates the minimum number of consecutive classes the student must attend to reach a required attendance percentage.

2. Objectives

- Calculate attendance percentage using attended and conducted classes.
- Use arithmetic operators and conditional statements.
- Use a loop to find the minimum additional classes needed.
- Display a clear result for eligible and not-yet-eligible students.

3. Assumptions

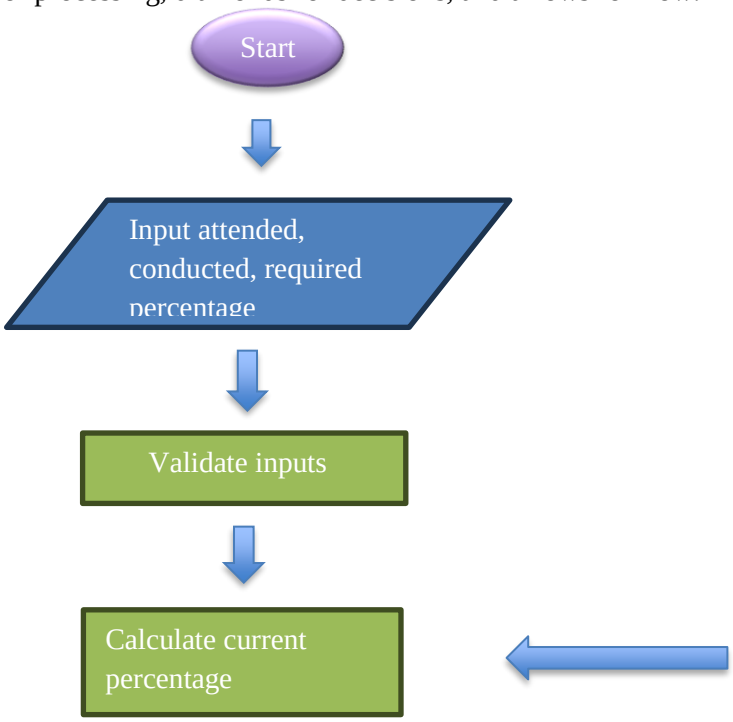
1. The number of classes conducted is greater than zero.
2. Classes attended are between zero and classes conducted.
3. The required attendance percentage is between 0 and 100.
4. The student attends every additional class considered in the recovery calculation.

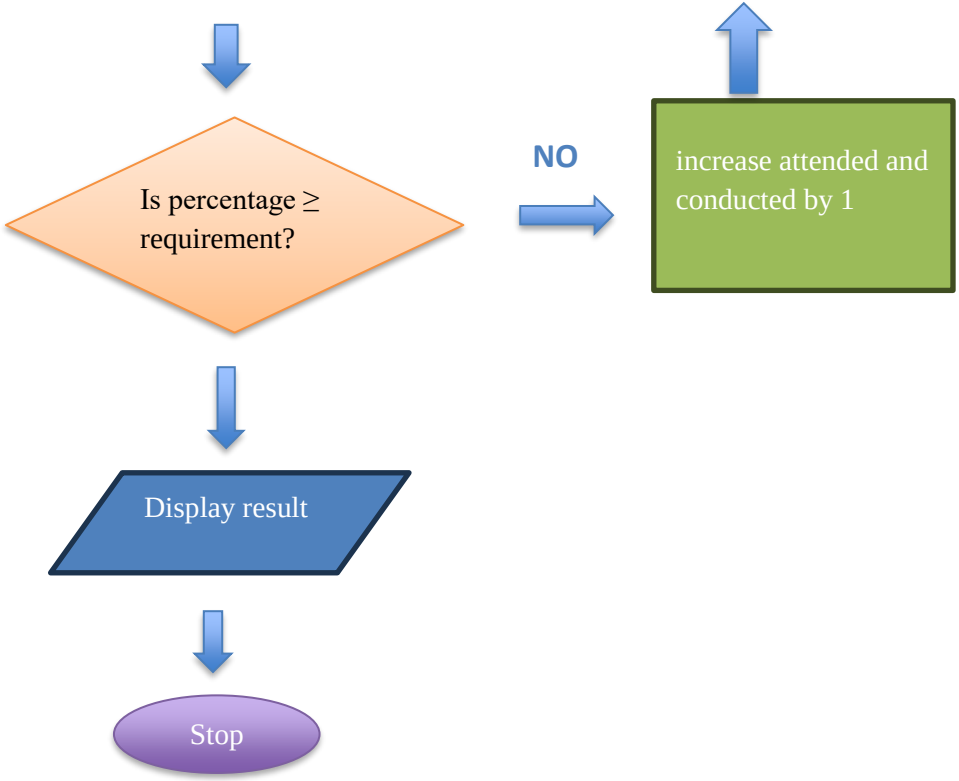
4. Algorithm

5. Start.
6. Input classes conducted, classes attended, and required percentage.
7. Validate the inputs.
8. Calculate current percentage = attended / conducted × 100.
9. If current percentage meets the requirement, display that the student is already eligible.
10. Otherwise, repeatedly add one attended class and one conducted class until the required percentage is reached.
11. Display current percentage and additional classes needed.
12. Stop.

5. Flowchart

Draw this flow in Word using Insert → Shapes. Use ovals for Start/Stop, parallelograms for input/output, rectangles for processing, diamonds for decisions, and arrows for flow.





6. Pseudocode

```
START
INPUT conducted, attended, required
IF conducted <= 0 OR attended < 0 OR attended > conducted OR required < 0 OR required > 100 THEN
    DISPLAY "Invalid input"
ELSE
    percentage = attended / conducted * 100
    additional = 0
    WHILE percentage < required
        attended = attended + 1
        conducted = conducted + 1
        additional = additional + 1
        percentage = attended / conducted * 100
    END WHILE
    DISPLAY percentage and additional
END IF
STOP
```

7. Python Source Code

```
conducted = int(input("Classes conducted: "))
attended = int(input("Classes attended: "))
required = float(input("Required attendance percentage: "))

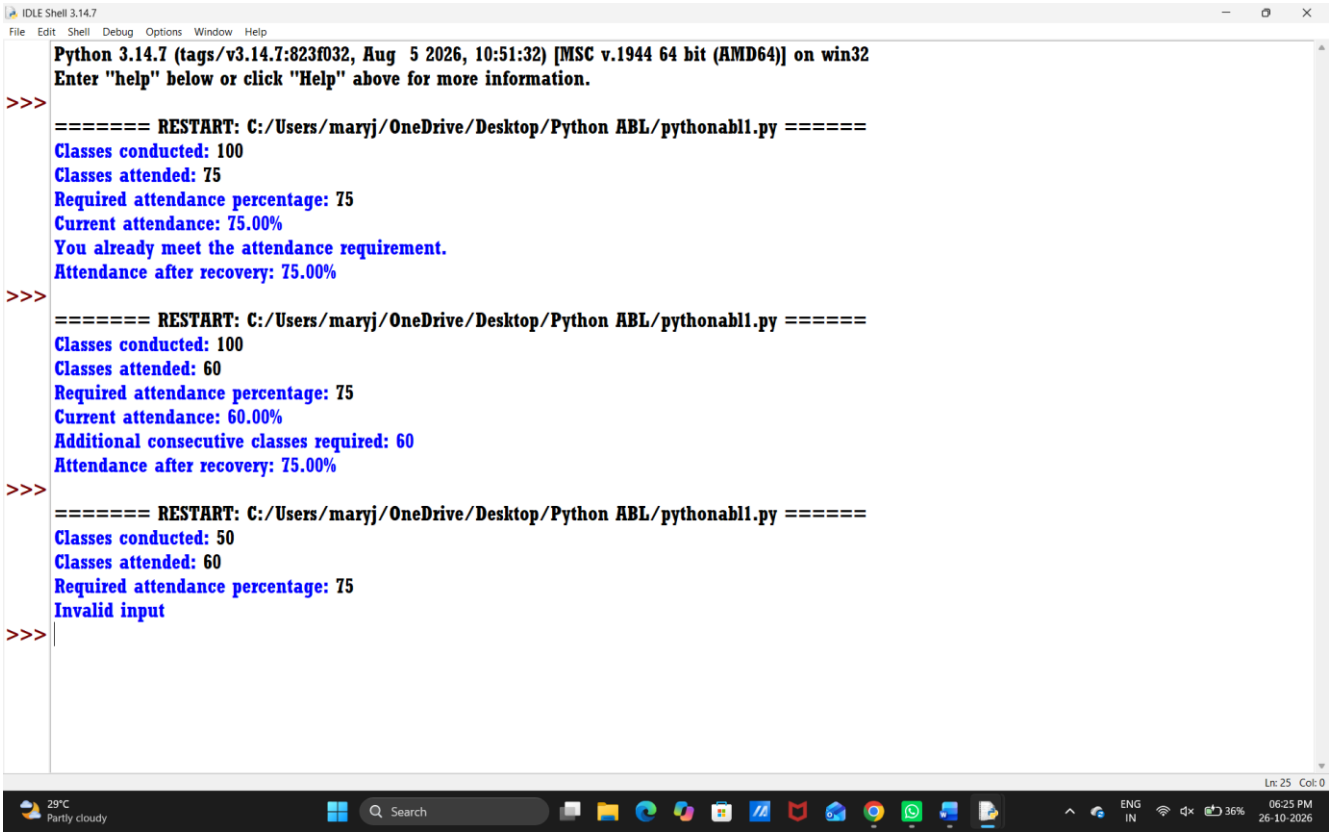
if (conducted <= 0 or attended < 0 or attended > conducted
        or required < 0 or required > 100):
    print("Invalid input")
else:
    current = attended / conducted * 100
    print(f"Current attendance: {current:.2f}%")

    additional = 0
    while current < required:
        attended += 1
```

```
        conducted += 1
        additional += 1
        current = attended / conducted * 100

    if additional == 0:
        print("You already meet the attendance requirement.")
    else:
        print("Additional consecutive classes required:", additional)
    print(f"Attendance after recovery: {current:.2f}%")
```

8. Test Cases and Output Screenshots



9. Conclusion

The program calculates current attendance and uses a while loop to estimate the minimum additional consecutive classes required to reach the specified attendance threshold.

CASE STUDY 2: Student Performance Trend Analysis

1. Problem Statement

Write a Python program to compare a student's scores across multiple assessments, calculate improvement or decline, and identify the subject with the lowest score.

2. Objectives

- Store subject names and marks in lists.
- Compare scores using loops and conditions.
- Calculate score changes between assessments.
- Identify the weakest subject.

3. Assumptions

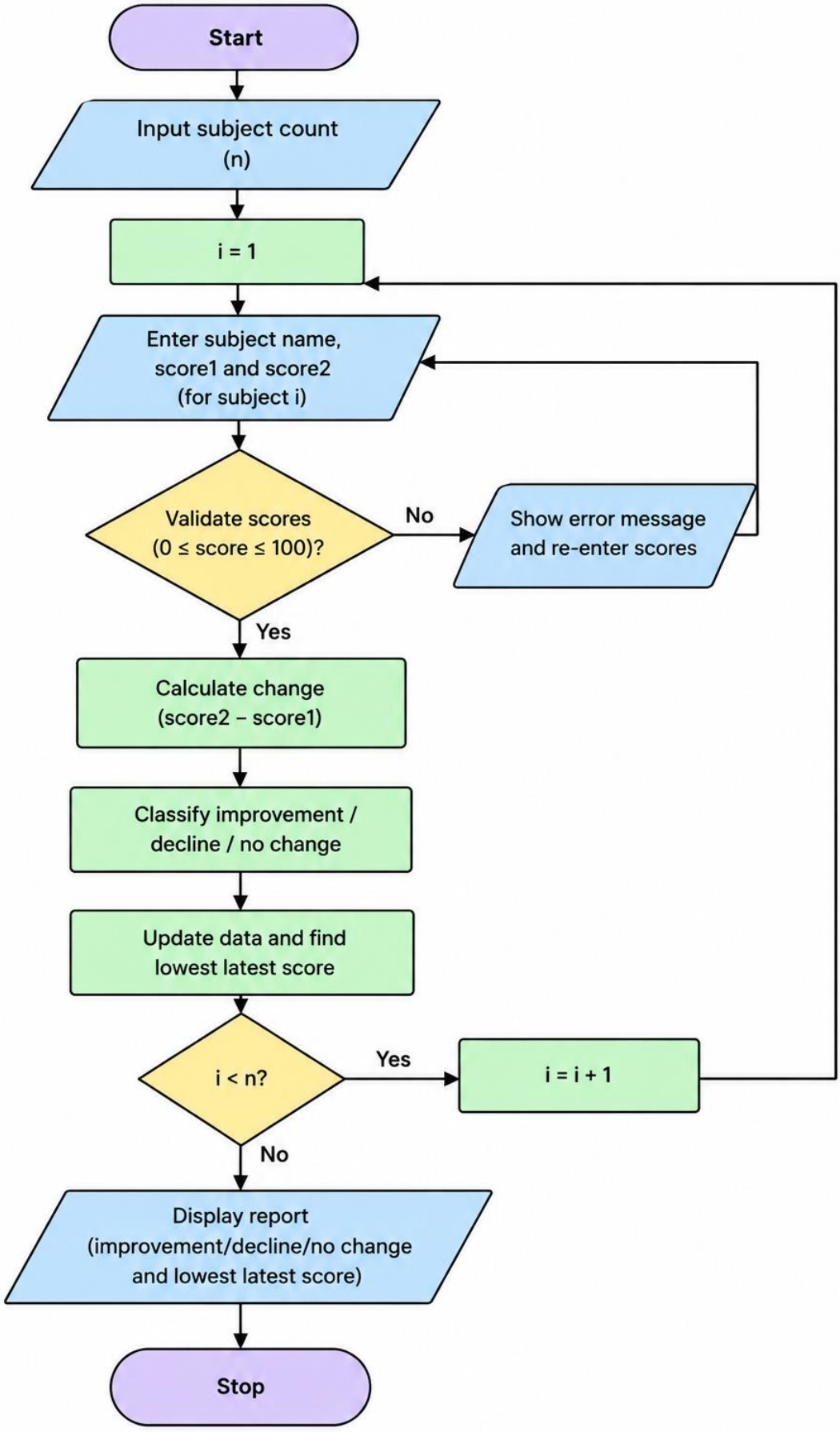
13. The same subjects are used for both assessments.
14. Marks are from 0 to 100.
15. At least one subject is entered.

4. Algorithm

16. Start.
17. Input the number of subjects.
18. For each subject, input its name, earlier score, and latest score.
19. Validate that both scores are between 0 and 100.
20. Calculate change = latest score − earlier score.
21. Classify the result as improved, declined, or unchanged.
22. Find the subject with the lowest latest score.
23. Display the comparison and weakest subject.
24. Stop.

5. Flowchart

Draw this flow in Word using Insert → Shapes. Use ovals for Start/Stop, parallelograms for input/output, rectangles for processing, diamonds for decisions, and arrows for flow.


Start
Input subject count (n)
i = 1
Enter subject name, score1 and score2 (for subject i)
Validate scores (0 ≤ score ≤ 100)?
No
Show error message and re-enter scores
Yes
Calculate change (score2 – score1)
Classify improvement / decline / no change
Update data and find lowest latest score
i < n?
Yes
i = i + 1
No
Display report (improvement/decline/no change and lowest latest score)
Stop

6. Pseudocode

```
START
INPUT n
CREATE lists for subjects, earlier scores, latest scores
FOR each subject from 1 to n
    INPUT subject, earlier, latest
    IF scores are outside 0..100 THEN DISPLAY invalid input and stop
    STORE values
    change = latest - earlier
    DISPLAY change category
END FOR
FIND subject with minimum latest score
DISPLAY weakest subject
STOP
```

7. Python Source Code

```
n = int(input("Number of subjects: "))
subjects = []
earlier_scores = []
latest_scores = []

if n <= 0:
    print("Enter at least one subject.")
else:
    valid = True
    for i in range(n):
        subject = input(f"Subject {i + 1} name: ")
        earlier = float(input("Earlier score (0-100): "))
        latest = float(input("Latest score (0-100): "))

        if not (0 <= earlier <= 100 and 0 <= latest <= 100):
            print("Invalid score. Restart the program with valid scores.")
            valid = False
            break

        subjects.append(subject)
        earlier_scores.append(earlier)
        latest_scores.append(latest)

    if valid:
        print("\nPerformance report")
        for i in range(n):
            change = latest_scores[i] - earlier_scores[i]
            if change > 0:
                status = "Improved"
            elif change < 0:
                status = "Declined"
            else:
                status = "Unchanged"
            print(f"{subjects[i]}: {change:+.1f} marks ({status})")

        lowest_index = latest_scores.index(min(latest_scores))
        print("Lowest-scoring subject:", subjects[lowest_index])
```

8. Test Cases and Output Screenshots



9. Conclusion

The program uses lists and loops to compare assessment results, describe score trends, and identify the subject with the lowest latest score.

CASE STUDY 3: Advanced Numeric Pattern Generator

1. Problem Statement

Write a Python program to print a symmetric hollow diamond pattern for a user-specified odd width. The program must reject invalid dimensions.

2. Objectives

- Practise nested loops.
- Understand spaces and character positions in patterns.
- Validate user input.
- Generate a symmetric hollow diamond.

3. Assumptions

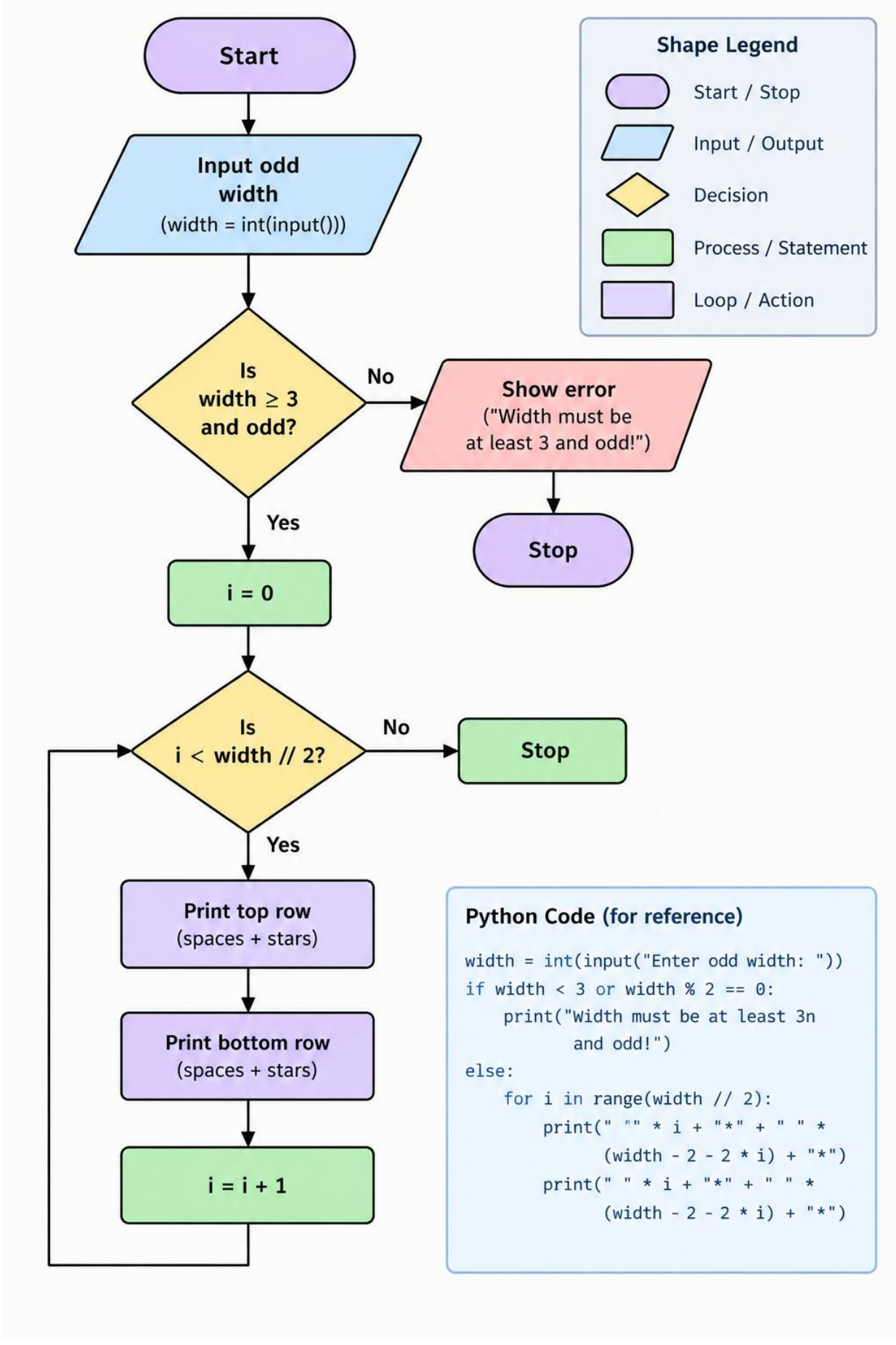
25. The width must be an odd integer of at least 3.
26. The pattern uses asterisks as boundary characters.
27. The pattern height equals the width.

4. Algorithm

28. Start.
29. Input an odd width.
30. If the width is less than 3 or even, display an error.
31. Set half = width // 2.
32. Generate the upper half including the middle row using nested loops.
33. Generate the lower half in reverse order.
34. Print asterisks only at the diamond boundary.
35. Stop.

5. Flowchart

Draw this flow in Word using Insert → Shapes. Use ovals for Start/Stop, parallelograms for input/output, rectangles for processing, diamonds for decisions, and arrows for flow.



Python Code (for reference)

```
width = int(input("Enter odd width: "))
if width < 3 or width % 2 == 0:
    print("Width must be at least 3n
          and odd!")
else:
    for i in range(width // 2):
        print(" " * i + "*" + " " *
              (width - 2 - 2 * i) + "*")
        print(" " * i + "*" + " " *
              (width - 2 - 2 * i) + "*")
```

6. Pseudocode

```
START
INPUT width
IF width < 3 OR width is even THEN
    DISPLAY "Width must be an odd integer of at least 3"
ELSE
    FOR row = 0 TO width - 1
        distance = row if row <= width//2 ELSE width-1-row
        leading_spaces = width//2 - distance
        PRINT leading_spaces
        IF distance = 0 THEN PRINT "*"
        ELSE PRINT "*", inner spaces, "*"
    END FOR
END IF
STOP
```

7. Python Source Code

```
width = int(input("Enter an odd width (3 or more): "))

if width < 3 or width % 2 == 0:
    print("Width must be an odd integer of at least 3.")
else:
    middle = width // 2
    for row in range(width):
        distance = row if row <= middle else width - 1 - row
        spaces = middle - distance
        print(" " * spaces, end="")

        if distance == 0:
            print("*")
        else:
            inner_spaces = 2 * distance - 1
            print("*" + " " * inner_spaces + "*")
```

8. Test Cases and Output Screenshots

Input / setup: width = 5

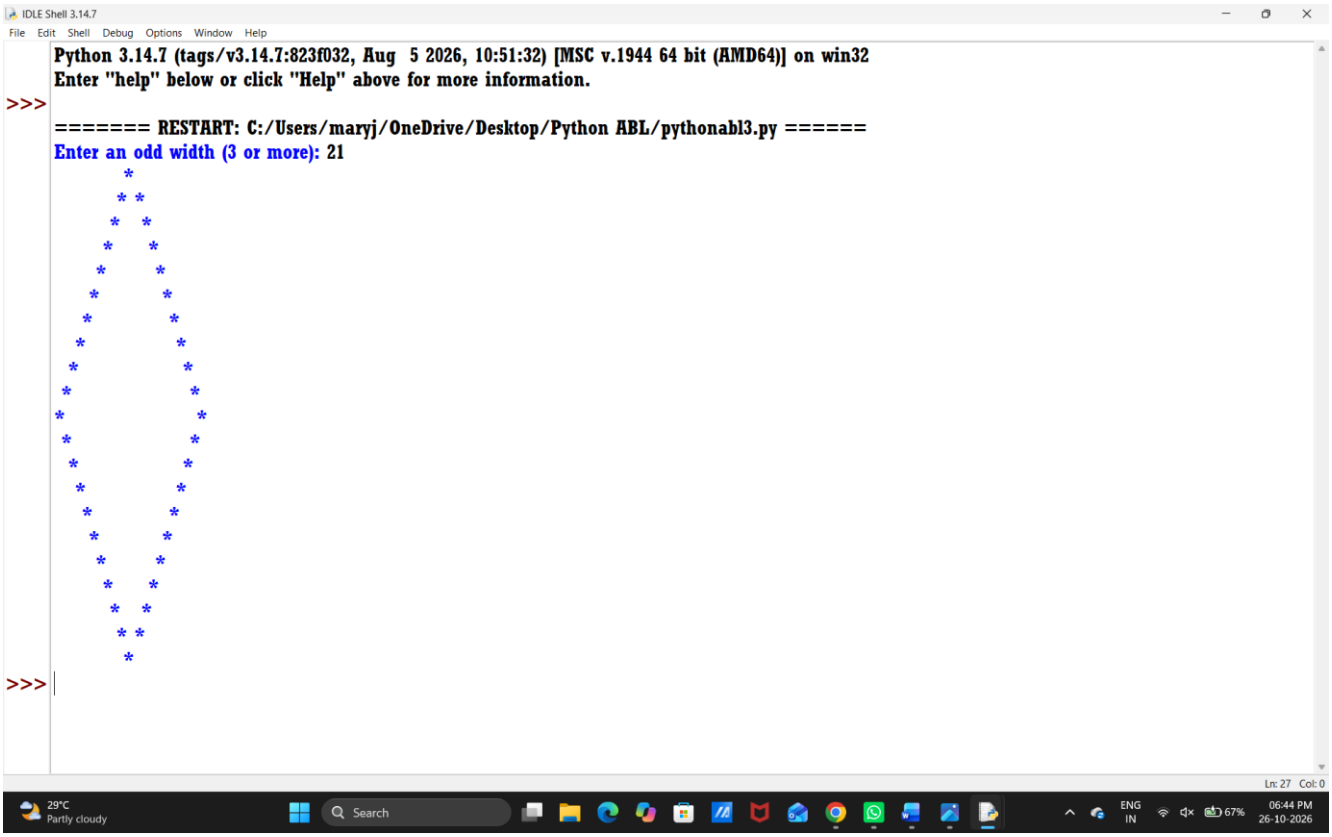
Expected output: *\n * *\n * *\n * *\n *

Test note: Expected pattern shown; terminal font affects visual alignment

Input / setup: width = 4

Expected output: Width must be an odd integer of at least 3.

Test note: Invalid even width



9. Conclusion

The program uses nested decisions, loops, and string repetition to produce a hollow diamond and validates the requested width.

CASE STUDY 4: Password Policy Validation Engine

1. Problem Statement

Write a Python program that checks a sample password against basic rules: minimum length, at least one uppercase letter, one lowercase letter, one digit, and one special character.

2. Objectives

- Practise string traversal.
- Use built-in string checks and conditions.
- Report each rule that fails.
- Understand simple password validation.

3. Assumptions

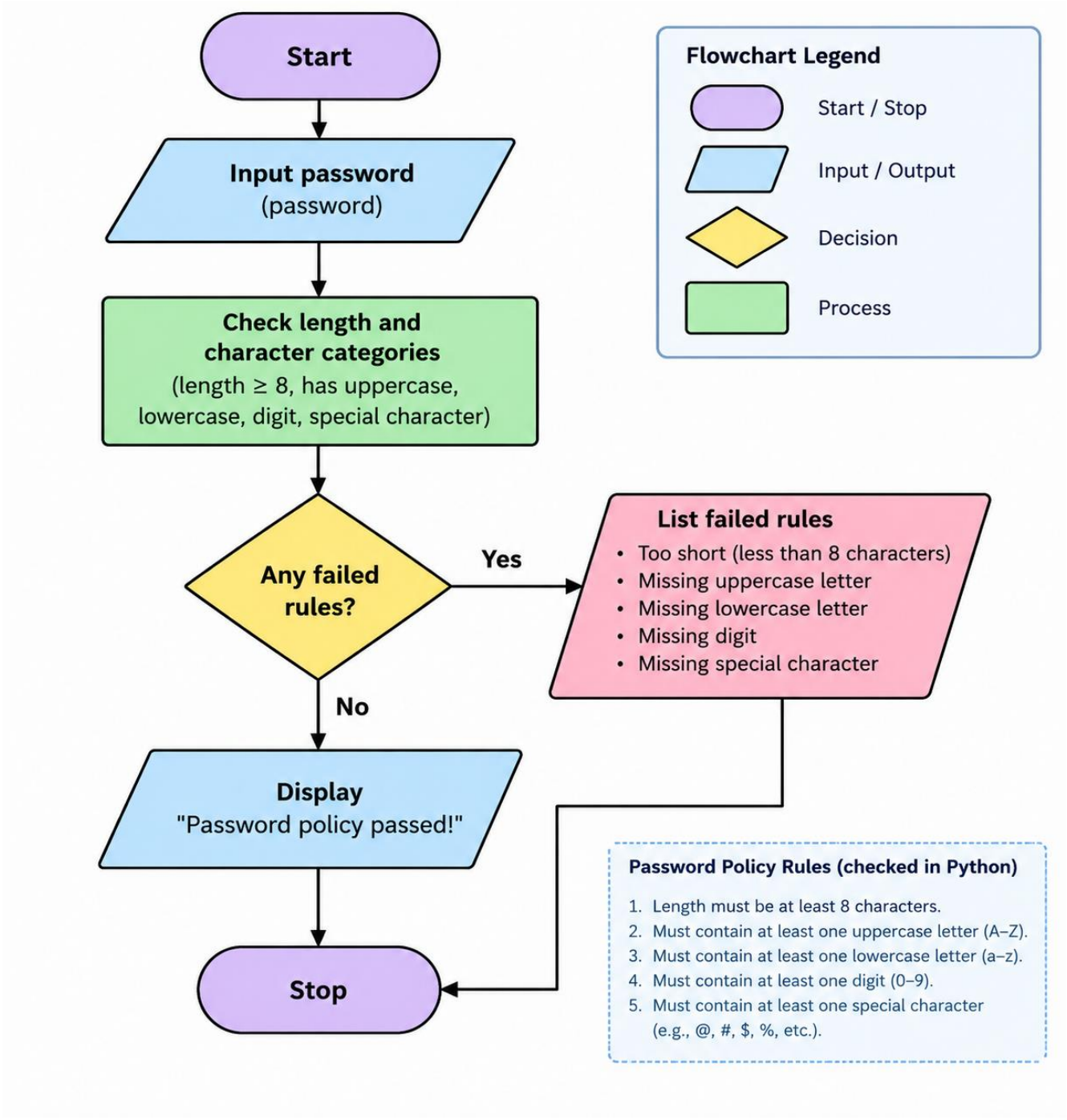
36. The minimum length is 8 characters for this exercise.
37. A special character is any character that is not a letter or digit.
38. This is a classroom exercise, not a complete security assessment.

4. Algorithm

39. Start.
40. Input a password.
41. Check its length.
42. Scan each character and record whether uppercase, lowercase, digit, and special characters are present.
43. Display every failed rule.
44. If no rule fails, display that the sample password meets the classroom policy.
45. Stop.

5. Flowchart

Draw this flow in Word using Insert → Shapes. Use ovals for Start/Stop, parallelograms for input/output, rectangles for processing, diamonds for decisions, and arrows for flow.



6. Pseudocode

```
START
INPUT password
SET failures = empty list
IF length(password) < 8 THEN add length rule to failures
SET upper, lower, digit, special = false
FOR each character in password
    UPDATE category flags
END FOR
IF any category flag is false THEN add its rule to failures
```

```
IF failures is empty THEN DISPLAY "Policy passed"
ELSE DISPLAY failures
STOP
```

7. Python Source Code

```
password = input("Enter a sample password: ")

failures = []
if len(password) < 8:
    failures.append("Use at least 8 characters.")

has_upper = False
has_lower = False
has_digit = False
has_special = False

for ch in password:
    if ch.isupper():
        has_upper = True
    elif ch.islower():
        has_lower = True
    elif ch.isdigit():
        has_digit = True
    else:
        has_special = True

if not has_upper:
    failures.append("Include an uppercase letter.")
if not has_lower:
    failures.append("Include a lowercase letter.")
if not has_digit:
    failures.append("Include a digit.")
if not has_special:
    failures.append("Include a special character.")

if len(failures) == 0:
    print("Password meets the classroom policy.")
else:
    print("Password does not meet the policy:")
    for rule in failures:
        print("-", rule)
```

8. Test Cases and Output Screenshots

Input / setup: Abcdef1!

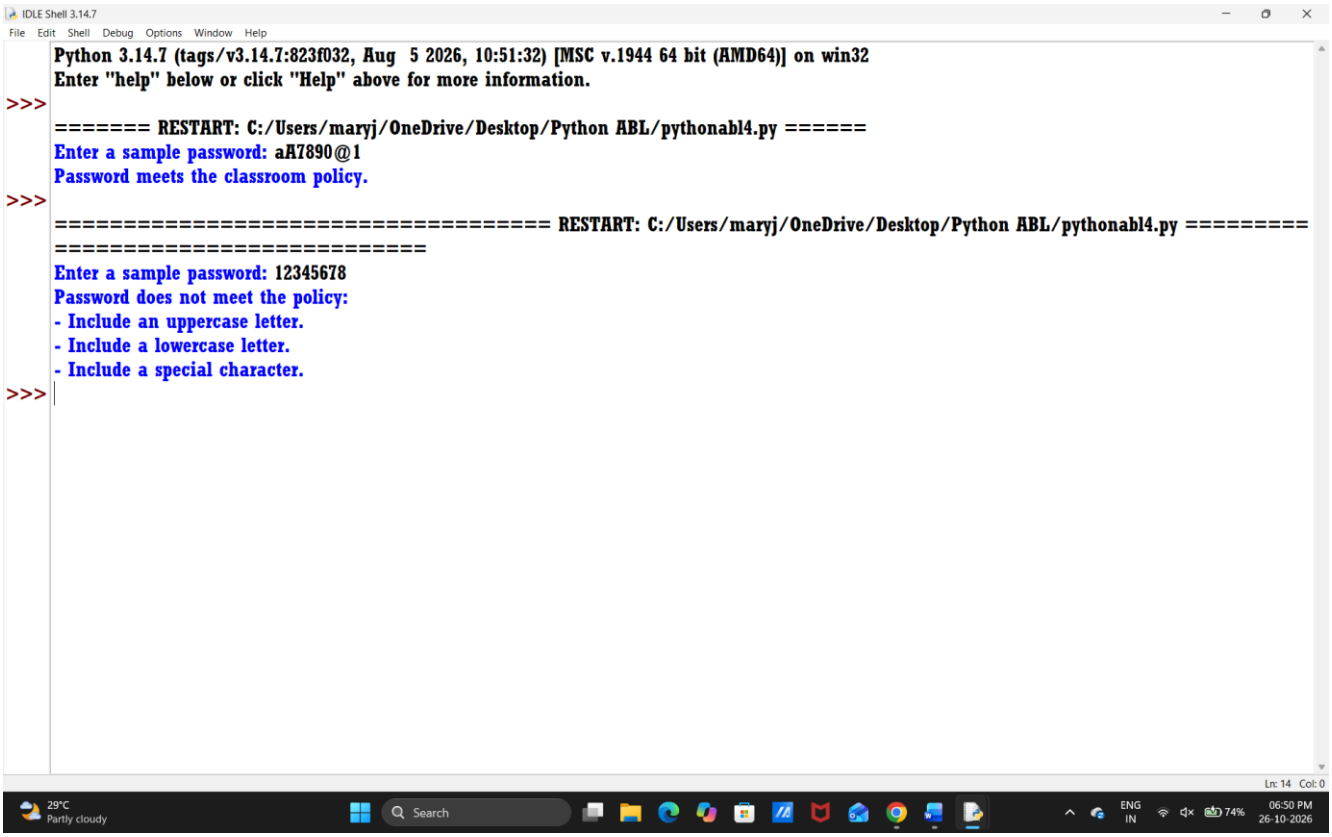
Expected output: Password meets the classroom policy.

Test note: All rules satisfied

Input / setup: abcdefg

Expected output: Does not meet policy: length, uppercase, digit, special character.

Test note: Several rules fail



9. Conclusion

The program checks a sample password using string operations and conditions, and explains which classroom rules are not satisfied.

CASE STUDY 5: Text Compression Using Character Runs

1. Problem Statement

Write a Python program that compresses consecutive repeated characters by recording each character followed by its run length, and compare the original and encoded lengths.

2. Objectives

- Traverse a string using a loop.
- Detect consecutive repeated characters.
- Build a new encoded string.
- Compare original and encoded lengths.

3. Assumptions

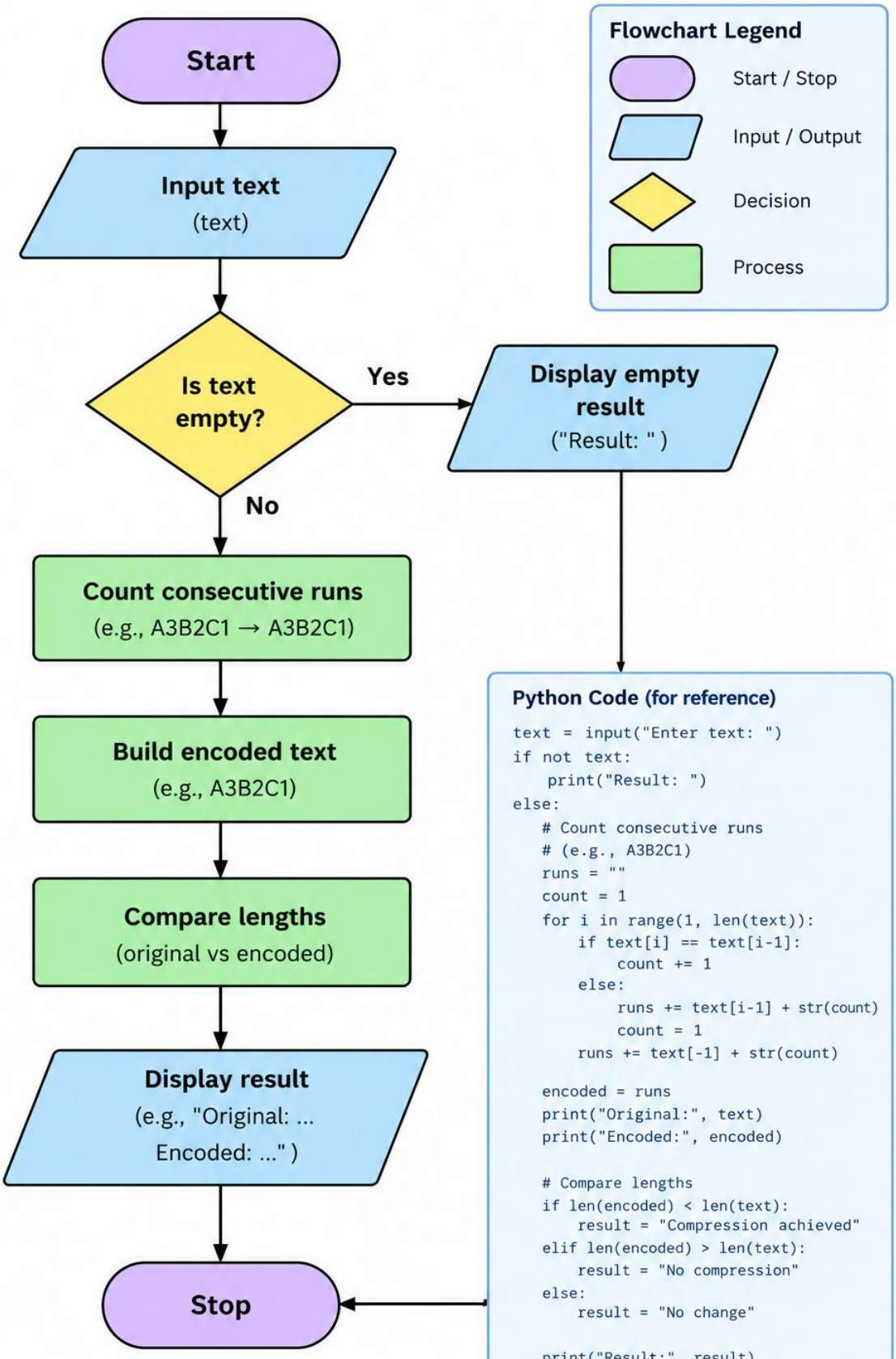
46. The input is a single line of text.
47. The program uses simple run-length encoding.
48. If the encoded text is not shorter, the program reports that compression is not beneficial.

4. Algorithm

49. Start.
50. Input a string.
51. If it is empty, display an empty-string message.
52. Set the current character and count.
53. Scan remaining characters; if the next character matches, increase count; otherwise append character and count to encoded text and reset count.
54. Append the final run.
55. Display original and encoded strings and their lengths.
56. Report whether encoding is shorter.
57. Stop.

5. Flowchart

Draw this flow in Word using Insert → Shapes. Use ovals for Start/Stop, parallelograms for input/output, rectangles for processing, diamonds for decisions, and arrows for flow.


Start
Input text
(text)
Is text empty?
Yes
Display empty result
("Result: ")
No
Count consecutive runs
(e.g., A3B2C1 → A3B2C1)
Build encoded text
(e.g., A3B2C1)
Compare lengths
(original vs encoded)
Display result
(e.g., "Original: ...
Encoded: ...")
Stop
Flowchart Legend
Start / Stop
Input / Output
Decision
Process
Python Code (for reference)
text = input("Enter text: ")
if not text:
print("Result: ")
else:
Count consecutive runs
(e.g., A3B2C1)
runs = ""
count = 1
for i in range(1, len(text)):
if text[i] == text[i-1]:
count += 1
else:
runs += text[i-1] + str(count)
count = 1
runs += text[-1] + str(count)
encoded = runs
print("Original:", text)
print("Encoded:", encoded)
Compare lengths
if len(encoded) < len(text):
result = "Compression achieved"
elif len(encoded) > len(text):
result = "No compression"
else:
result = "No change"
print("Result:", result)

6. Pseudocode

```
START
INPUT text
IF text is empty THEN DISPLAY empty message
ELSE
    encoded = ""
    count = 1
    FOR each next character
        IF next character equals current THEN count = count + 1
        ELSE append current and count; set current = next; count = 1
    END FOR
    append final character and count
    DISPLAY encoded and lengths
END IF
STOP
```

7. Python Source Code

```
text = input("Enter text to compress: ")

if text == "":
    print("The input is empty.")
else:
    encoded = ""
    count = 1

    for i in range(1, len(text)):
        if text[i] == text[i - 1]:
            count += 1
        else:
            encoded += text[i - 1] + str(count)
            count = 1

    encoded += text[-1] + str(count)

    print("Original text:", text)
    print("Encoded text:", encoded)
    print("Original length:", len(text))
    print("Encoded length:", len(encoded))

    if len(encoded) < len(text):
        print("Encoding is shorter.")
    else:
        print("Encoding is not beneficial for this input.")
```

8. Test Cases and Output Screenshots

Input / setup: aaabbc

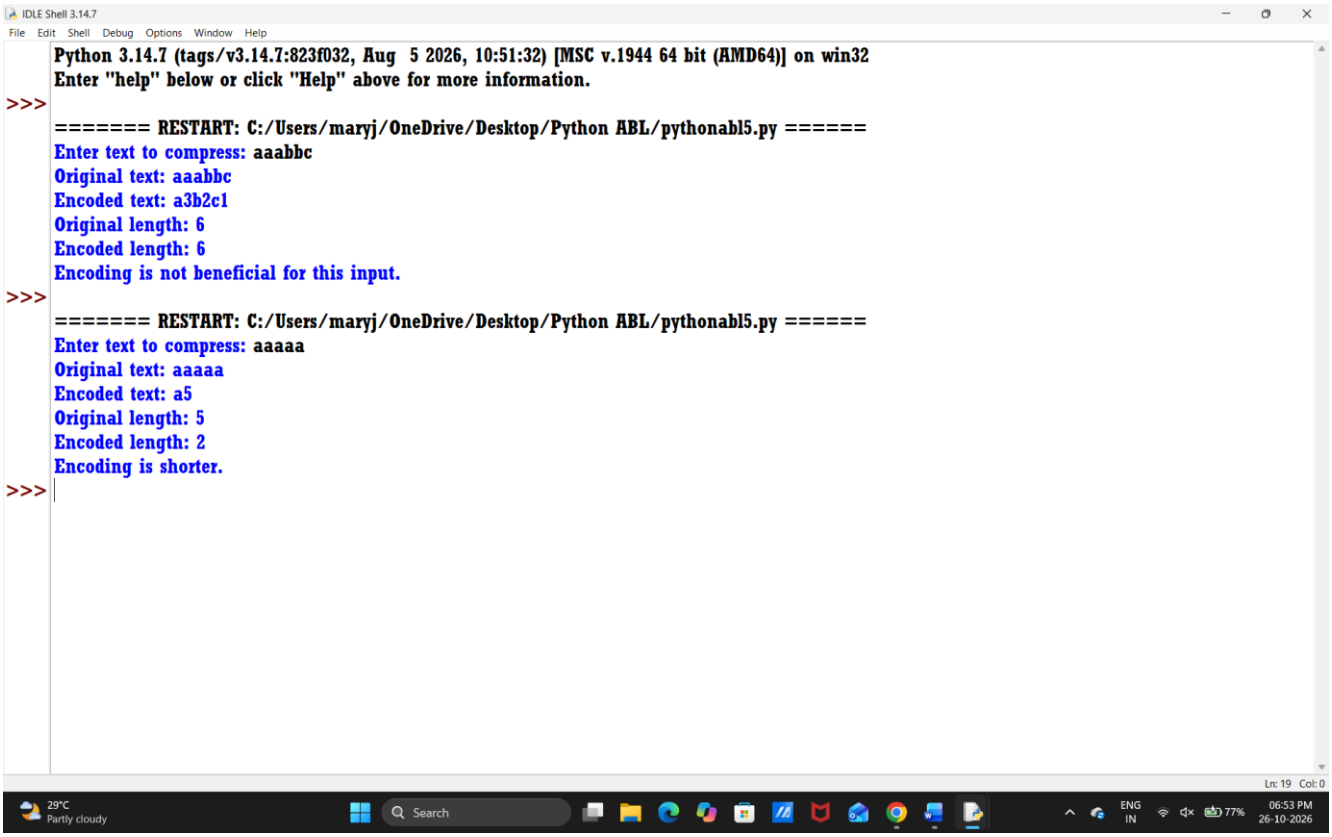
Expected output: a3b2c1; original length 6; encoded length 6

Test note: Encoding is not beneficial

Input / setup: aaaaa

Expected output: a5; original length 5; encoded length 2

Test note: Encoding is shorter



9. Conclusion

The program groups consecutive repeated characters, creates a run-length representation, and compares the encoded length with the original text length.

CASE STUDY 6: Student Marks Matrix Analysis

1. Problem Statement

Write a Python program that stores marks for multiple students across multiple subjects in a nested list, calculates each student's average and each subject's average, and identifies the top student.

2. Objectives

- Represent tabular marks using nested lists.
- Use nested loops to process rows and columns.
- Calculate row-wise and column-wise averages.
- Identify the student with the highest average.

3. Assumptions

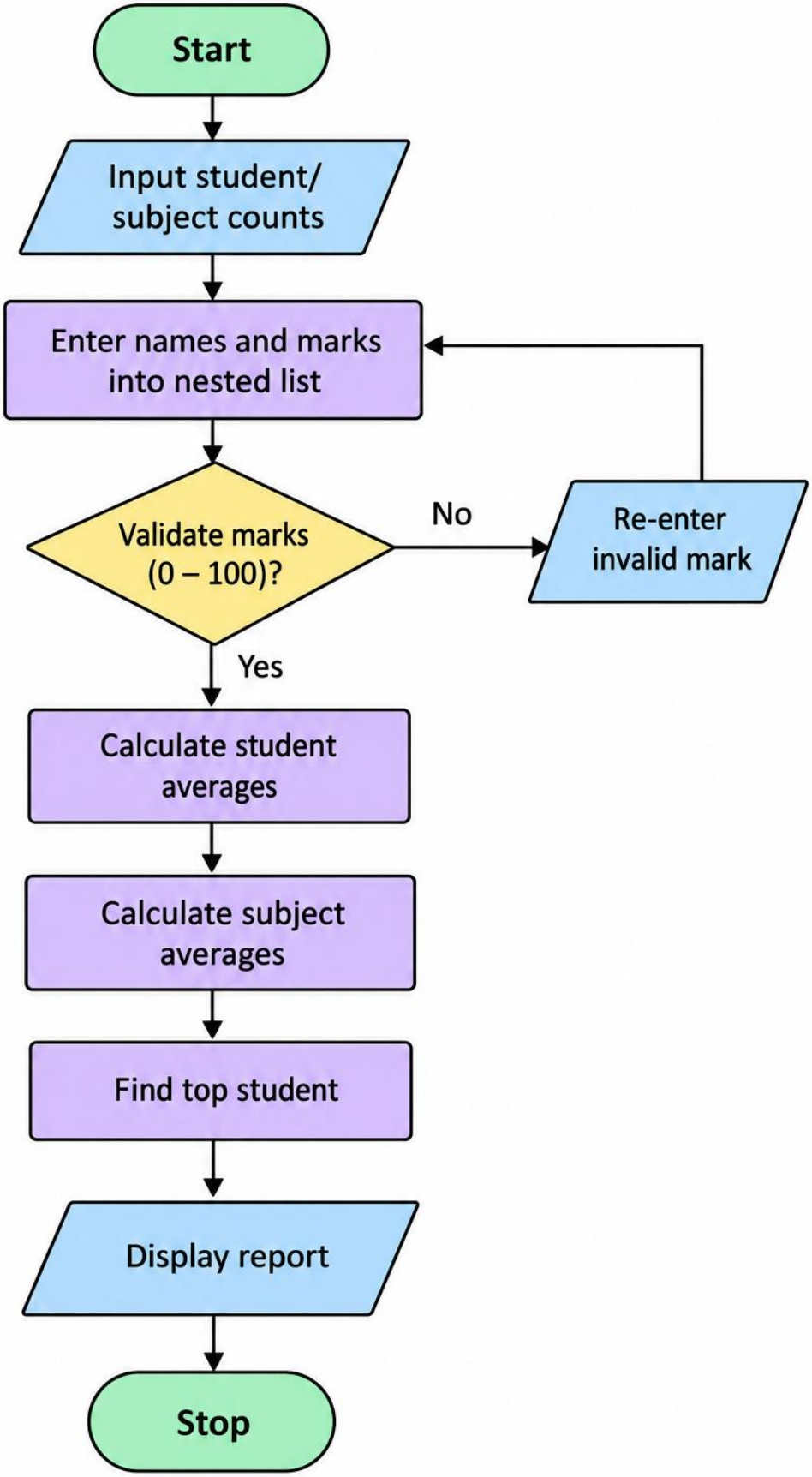
58. At least one student and one subject are entered.
59. Marks are between 0 and 100.
60. All students are assessed in the same subjects.

4. Algorithm

61. Start.
62. Input student count and subject count.
63. Input student names and marks, storing marks in a nested list.
64. Validate every mark.
65. Calculate each student's total and average.
66. Calculate each subject's average.
67. Find the student with the highest average.
68. Display averages and top student.
69. Stop.

5. Flowchart

Draw this flow in Word using Insert → Shapes. Use ovals for Start/Stop, parallelograms for input/output, rectangles for processing, diamonds for decisions, and arrows for flow.


Start
Input student/ subject counts
Enter names and marks into nested list
Validate marks (0 – 100)?
No
Re-enter invalid mark
Yes
Calculate student averages
Calculate subject averages
Find top student
Display report
Stop

6. Pseudocode

```
START
INPUT number_of_students, number_of_subjects
FOR each student
    INPUT student name
    FOR each subject
        INPUT mark
        VALIDATE mark from 0 to 100
        STORE mark in nested list
    END FOR
END FOR
CALCULATE each row average
CALCULATE each column average
FIND maximum student average
DISPLAY results
STOP
```

7. Python Source Code

```
students = []
marks_matrix = []

student_count = int(input("Number of students: "))
subject_count = int(input("Number of subjects: "))

if student_count <= 0 or subject_count <= 0:
    print("Counts must be greater than zero.")
else:
    valid = True
    for i in range(student_count):
        name = input(f"Student {i + 1} name: ")
        row = []
        for j in range(subject_count):
            mark = float(input(f"Mark for subject {j + 1}: "))
            if not 0 <= mark <= 100:
                print("Invalid mark. Restart with valid marks.")
                valid = False
                break
            row.append(mark)
        if not valid:
            break
        students.append(name)
        marks_matrix.append(row)

    if valid:
        averages = []
        for i in range(student_count):
            avg = sum(marks_matrix[i]) / subject_count
            averages.append(avg)
            print(f"{students[i]} average: {avg:.2f}")

        print("\nSubject averages:")
        for j in range(subject_count):
            total = 0
            for i in range(student_count):
                total += marks_matrix[i][j]
            print(f"Subject {j + 1}: {total / student_count:.2f}")
```

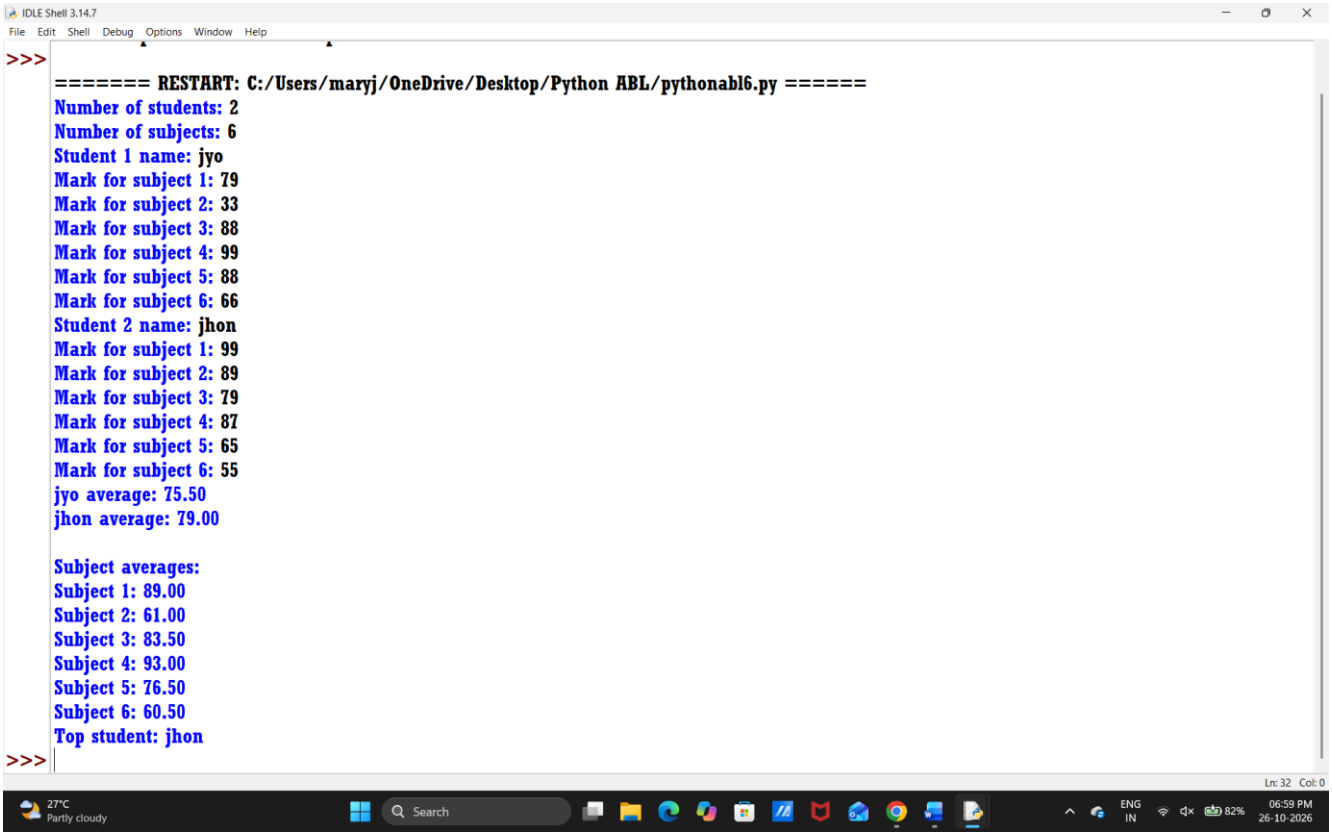
```
top = averages.index(max(averages))
print("Top student:", students[top])
```

8. Test Cases and Output Screenshots

Input / setup: Asha: 80,90; Ravi: 70,75

Expected output: Asha average 85.00; Ravi average 72.50; subject averages 75.00 and 82.50; top student Asha

Test note: Two students and two subjects



9. Conclusion

The program uses a nested list to store marks and nested loops to calculate student and subject averages, then identifies the top student.

CASE STUDY 7: Tuple-Based Product Catalogue Audit

1. Problem Statement

Write a Python program that stores product IDs, prices, and stock quantities as tuples, checks for invalid records, calculates inventory value, and identifies products that need replenishment.

2. Objectives

- Store fixed product records using tuples.
- Validate price and stock values.
- Calculate inventory value.
- Identify items below a stock threshold.

3. Assumptions

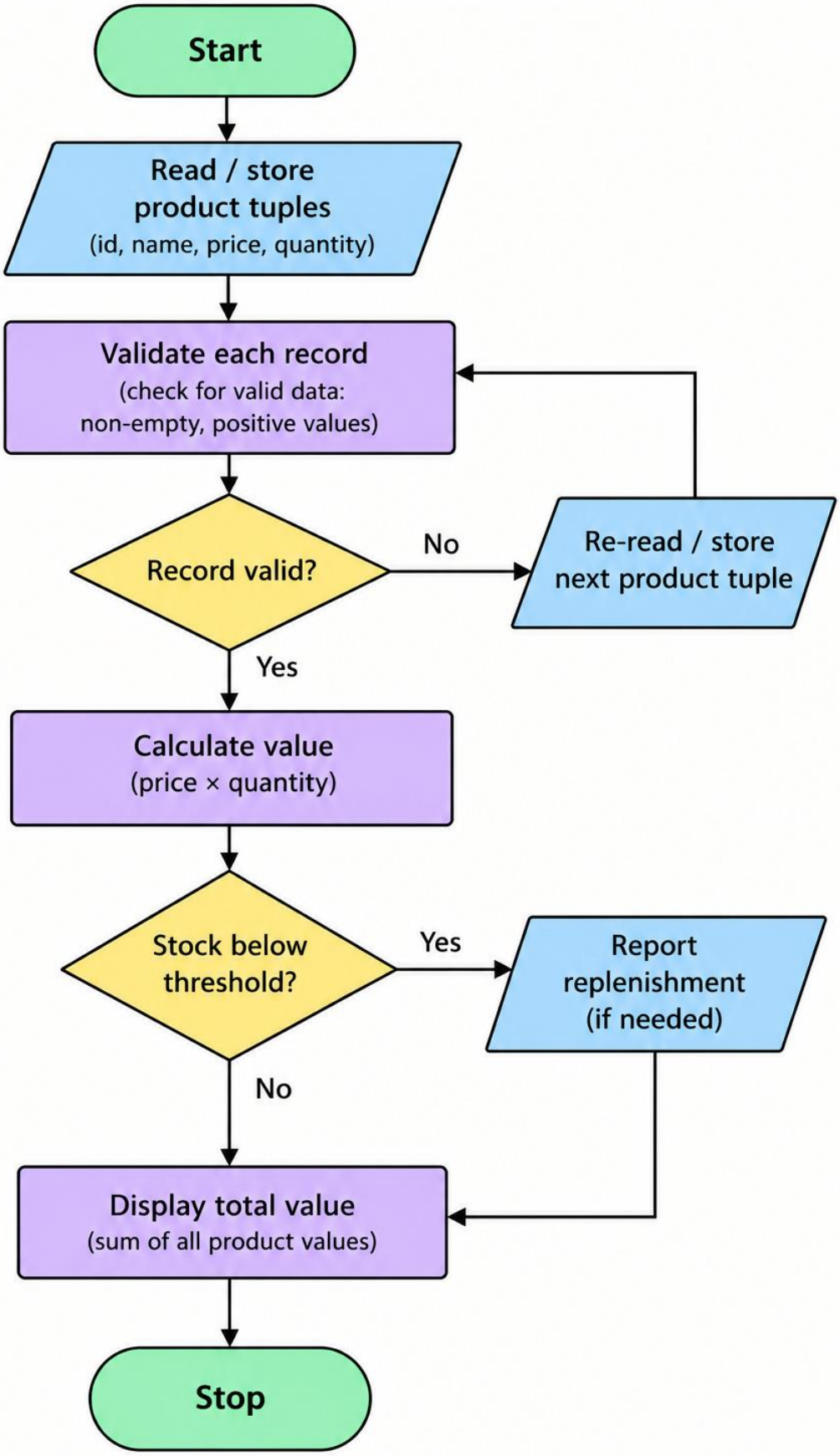
70. Each record contains product ID, price, and quantity.
71. Prices and quantities cannot be negative.
72. A product needs replenishment when stock is below 5 units.

4. Algorithm

73. Start.
74. Create a list of product tuples.
75. Set total inventory value to zero.
76. For each tuple, check price and quantity.
77. If valid, calculate price × quantity and add it to total.
78. If quantity is below 5, report replenishment needed.
79. Display total inventory value.
80. Stop.

5. Flowchart

Draw this flow in Word using Insert → Shapes. Use ovals for Start/Stop, parallelograms for input/output, rectangles for processing, diamonds for decisions, and arrows for flow.


Start
Read / store
product tuples
(id, name, price, quantity)
Validate each record
(check for valid data:
non-empty, positive values)
Record valid?
No
Re-read / store
next product tuple
Yes
Calculate value
(price × quantity)
Stock below
threshold?
Yes
Report
replenishment
(if needed)
No
Display total value
(sum of all product values)
Stop

Pseudocode

```
START
products = list of (product_id, price, quantity) tuples
total_value = 0
FOR each product tuple
    IF price < 0 OR quantity < 0 THEN DISPLAY invalid record
    ELSE
        value = price * quantity
        total_value = total_value + value
        IF quantity < 5 THEN DISPLAY replenishment required
    END IF
END FOR
DISPLAY total_value
STOP
```

7. Python Source Code

```
products = [
    ("P101", 25.50, 10),
    ("P102", 120.00, 3),
    ("P103", 45.00, 0)
]

total_value = 0

print("Product Catalogue Audit")
for product in products:
    product_id, price, quantity = product

    if price < 0 or quantity < 0:
        print(product_id, "has an invalid record.")
    else:
        value = price * quantity
        total_value += value
        print(f"{product_id}: stock value = Rs. {value:.2f}")
        if quantity < 5:
            print("  Replenishment required.")

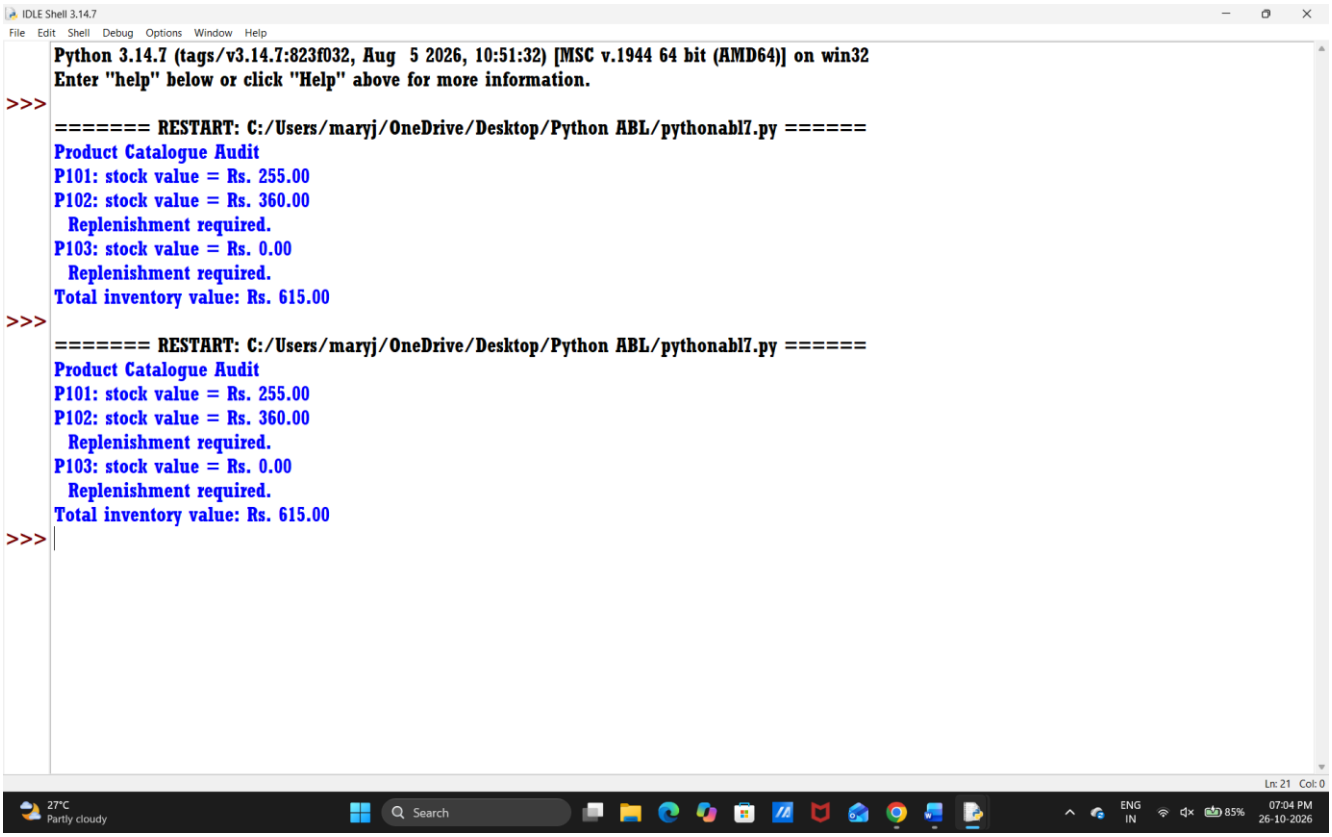
print(f"Total inventory value: Rs. {total_value:.2f}")
```

8. Test Cases and Output Screenshots

Input / setup: P101 (25.50,10); P102 (120,3); P103 (45,0)

Expected output: P101 value ₹255.00; P102 value ₹360.00 and replenishment; P103 value ₹0.00 and replenishment; total ₹615.00

Test note: Uses sample records built into the program



9. Conclusion

The program uses tuples and tuple unpacking to audit product records, calculate inventory value, and flag products with low stock.

CASE STUDY 8: Food Waste Monitoring System

1. Problem Statement

Write a Python program to analyse food prepared, food served, and leftovers across several days, calculate the waste percentage, and identify days exceeding an acceptable waste limit.

2. Objectives

- Store daily food quantities in lists.
- Calculate leftovers and waste percentages.
- Compare waste against a threshold.
- Identify the day with the highest waste percentage.

3. Assumptions

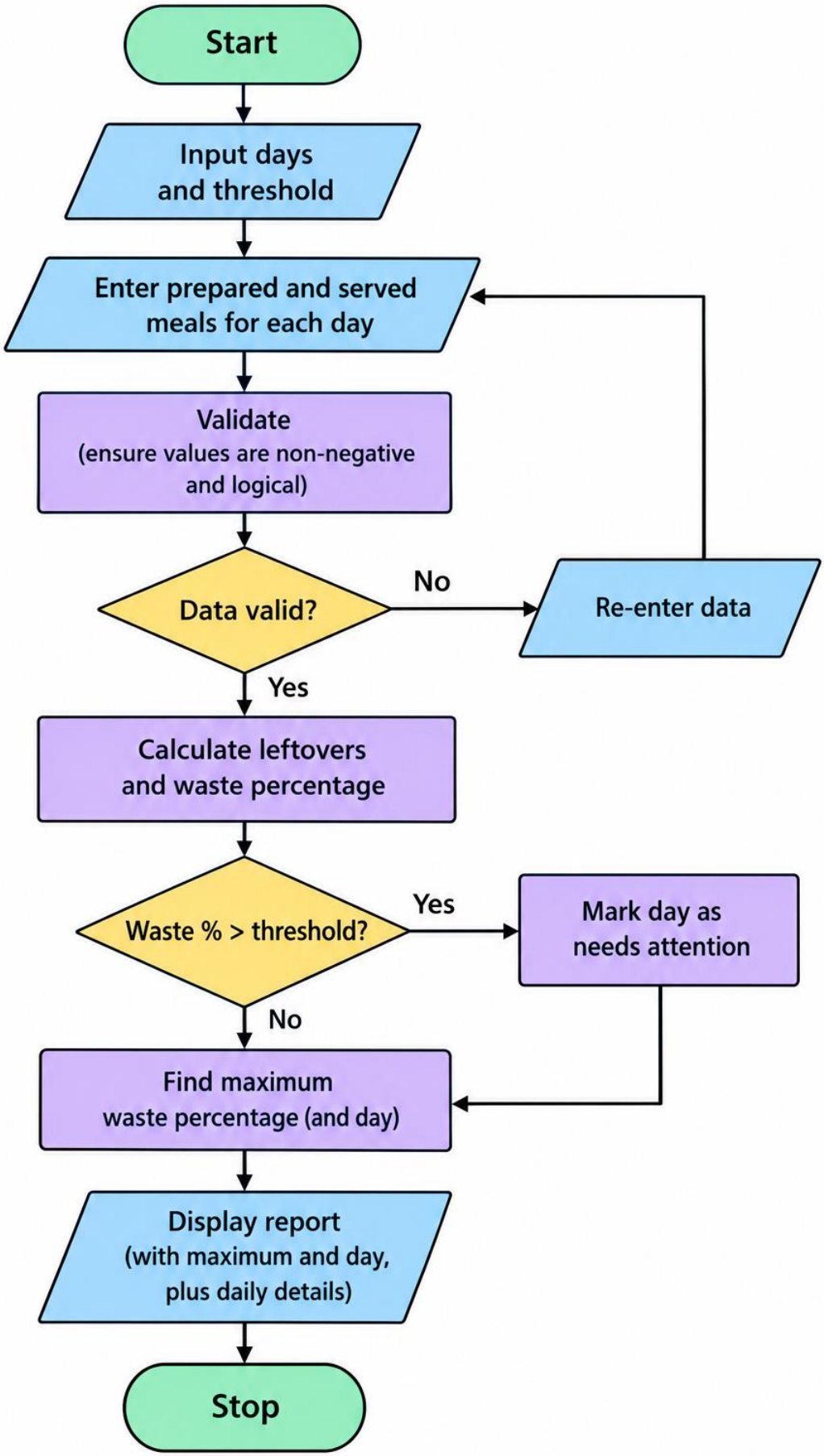
81. Quantities are entered in the same unit, such as meals.
82. Prepared quantity must be greater than zero.
83. Served meals must be between zero and prepared meals.
84. The acceptable waste limit is supplied as a percentage.

4. Algorithm

85. Start.
86. Input number of days and acceptable waste percentage.
87. For each day, input prepared and served meals.
88. Validate quantities.
89. Calculate leftovers = prepared − served.
90. Calculate waste percentage = leftovers / prepared × 100.
91. Flag days above the limit.
92. Find the day with the highest waste percentage.
93. Display daily report and highest-waste day.
94. Stop.

5. Flowchart

Draw this flow in Word using Insert → Shapes. Use ovals for Start/Stop, parallelograms for input/output, rectangles for processing, diamonds for decisions, and arrows for flow.


Start
Input days and threshold
Enter prepared and served meals for each day
Validate (ensure values are non-negative and logical)
Data valid?
No
Re-enter data
Yes
Calculate leftovers and waste percentage
Waste % > threshold?
Yes
Mark day as needs attention
No
Find maximum waste percentage (and day)
Display report (with maximum and day, plus daily details)
Stop

6. Pseudocode

```
START
INPUT days, acceptable_limit
FOR each day
    INPUT prepared, served
    VALIDATE prepared > 0 and 0 <= served <= prepared
    leftovers = prepared - served
    waste_percent = leftovers / prepared * 100
    IF waste_percent > acceptable_limit THEN FLAG day
    STORE result
END FOR
FIND day with maximum waste percentage
DISPLAY daily report and maximum-waste day
STOP
```

7. Python Source Code

```
days = int(input("Number of days: "))
limit = float(input("Acceptable waste percentage: "))

if days <= 0 or not 0 <= limit <= 100:
    print("Invalid number of days or waste limit.")
else:
    records = []
    valid = True

    for day in range(1, days + 1):
        prepared = int(input(f"Day {day} meals prepared: "))
        served = int(input(f"Day {day} meals served: "))

        if prepared <= 0 or served < 0 or served > prepared:
            print("Invalid quantities. Restart with valid data.")
            valid = False
            break

        leftovers = prepared - served
        waste = leftovers / prepared * 100
        records.append((day, prepared, served, leftovers, waste))

    if valid:
        for day, prepared, served, leftovers, waste in records:
            print(f"Day {day}: leftovers={leftovers}, waste={waste:.2f}%")
            if waste > limit:
                print("  Warning: waste exceeds the acceptable limit.")

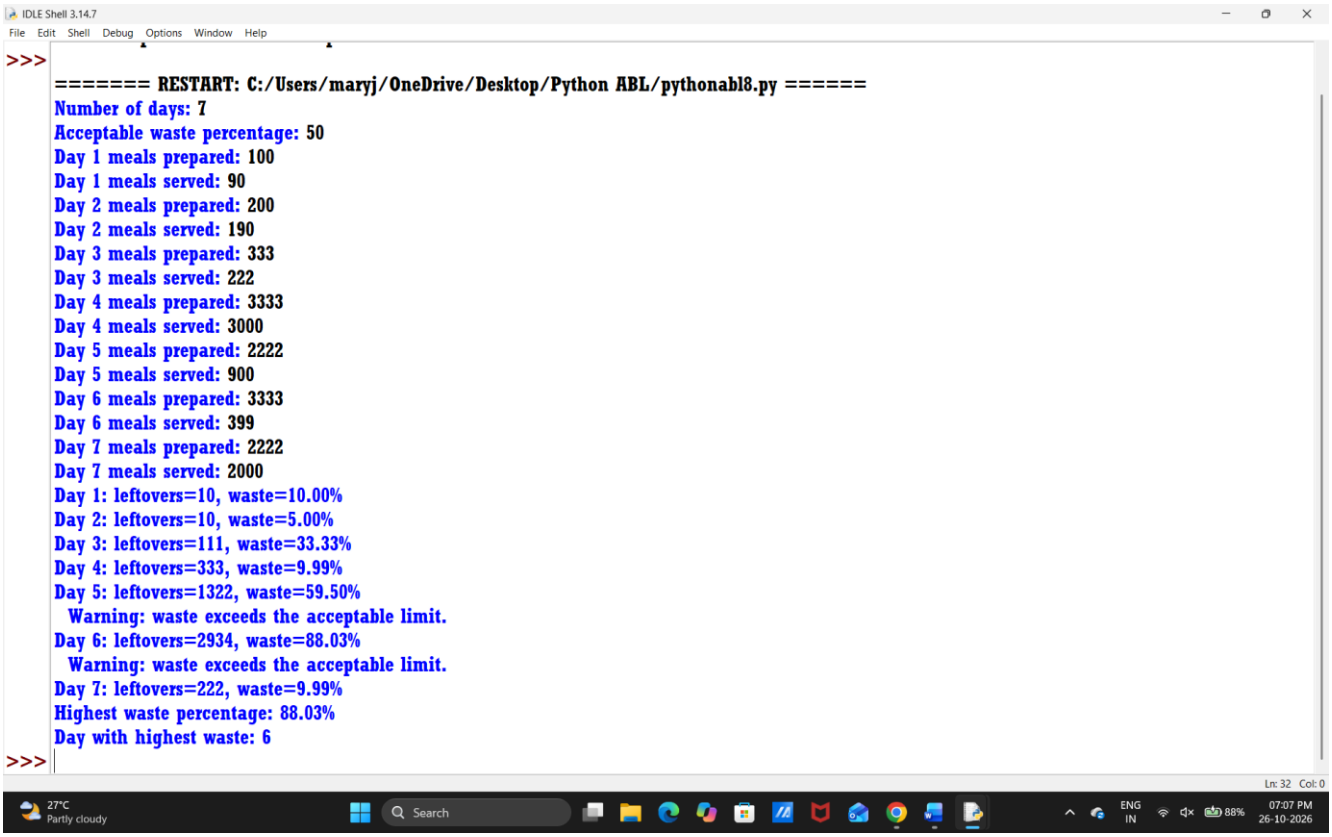
        highest = max(records, key=lambda record: record[4])
        print("Highest waste percentage:", f"{highest[4]:.2f}%")
        print("Day with highest waste:", highest[0])
```

8. Test Cases and Output Screenshots

Input / setup: Day 1 prepared 100, served 90; Day 2 prepared 80, served 60; limit 15%

Expected output: Day 1 leftovers 10, waste 10.00%; Day 2 leftovers 20, waste 25.00%, warning; highest waste Day 2

Test note: Two-day example



9. Conclusion

The program measures food waste over multiple days, flags waste above the chosen limit, and identifies the day with the highest waste percentage.

CASE STUDY 9: Multi-Product Price Fluctuation Analysis

1. Problem Statement

Write a Python program to compare historical and current prices for several products, calculate percentage changes, identify the largest increase or decrease, and flag changes above a given threshold.

2. Objectives

- Store product names and prices in lists.
- Calculate percentage price change.
- Classify price increases and decreases.
- Identify the largest absolute fluctuation.

3. Assumptions

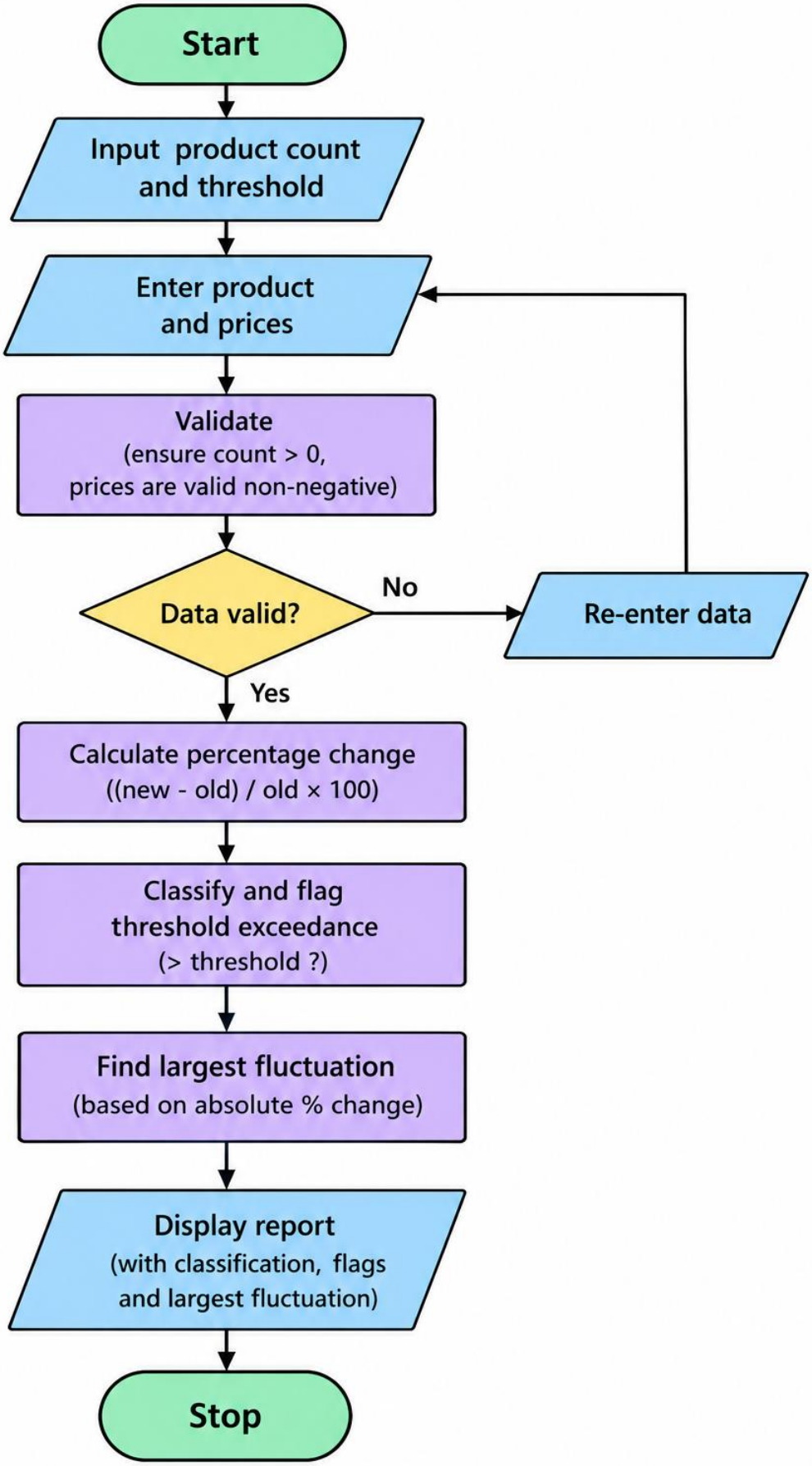
95. Historical prices are greater than zero.
96. Current prices are zero or greater.
97. The threshold is a non-negative percentage.
98. The same products are compared across both periods.

4. Algorithm

99. Start.
100. Input number of products and threshold percentage.
101. For each product, input its name, old price, and current price.
102. Validate the prices.
103. Calculate percentage change = (current − old) / old × 100.
104. Classify as increased, decreased, or unchanged.
105. Flag absolute changes above threshold.
106. Find the largest absolute percentage change.
107. Display report.
108. Stop.

5. Flowchart

Draw this flow in Word using Insert → Shapes. Use ovals for Start/Stop, parallelograms for input/output, rectangles for processing, diamonds for decisions, and arrows for flow.


Start
Input product count and threshold
Enter product and prices
Validate (ensure count > 0, prices are valid non-negative)
Data valid?
No
Re-enter data
Yes
Calculate percentage change ((new - old) / old × 100)
Classify and flag threshold exceedance (> threshold ?)
Find largest fluctuation (based on absolute % change)
Display report (with classification, flags and largest fluctuation)
Stop

6. Pseudocode

```
START
INPUT n, threshold
FOR each product
    INPUT name, old_price, current_price
    IF old_price <= 0 OR current_price < 0 THEN report invalid data
    ELSE
        change = (current_price - old_price) / old_price * 100
        CLASSIFY change
        IF ABS(change) > threshold THEN FLAG product
        STORE result
    END IF
END FOR
FIND largest absolute change among valid products
DISPLAY report
STOP
```

7. Python Source Code

```
n = int(input("Number of products: "))
threshold = float(input("Flag changes above (%) : "))

if n <= 0 or threshold < 0:
    print("Invalid product count or threshold.")
else:
    results = []
    valid = True

    for i in range(n):
        name = input(f"Product {i + 1} name: ")
        old_price = float(input("Historical price: "))
        current_price = float(input("Current price: "))

        if old_price <= 0 or current_price < 0:
            print("Invalid price. Restart with valid data.")
            valid = False
            break

        change = (current_price - old_price) / old_price * 100
        if change > 0:
            status = "Increased"
        elif change < 0:
            status = "Decreased"
        else:
            status = "Unchanged"

        results.append((name, change, status))

    if valid:
        for name, change, status in results:
            print(f"{name}: {change:+.2f}% ({status})")
            if abs(change) > threshold:
                print("  Alert: change exceeds threshold.")

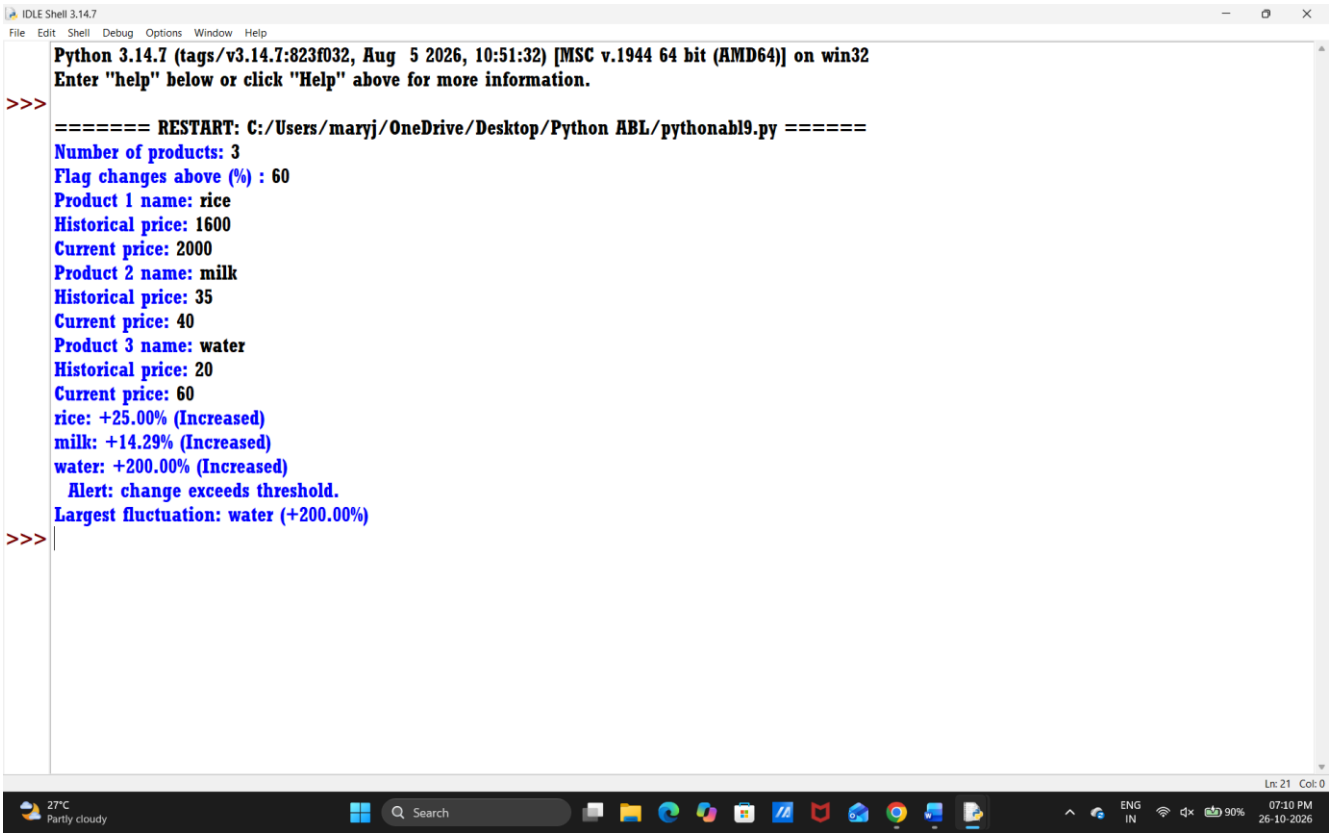
        largest = max(results, key=lambda item: abs(item[1]))
        print("Largest fluctuation:", largest[0],
              f"({largest[1]:+.2f}%)")
```

8. Test Cases and Output Screenshots

Input / setup: Rice old 50 current 60; Milk old 40 current 36; threshold 15%

Expected output: Rice +20.00% increased and flagged; Milk -10.00% decreased; largest fluctuation Rice

Test note: Two product example



9. Conclusion

The program compares prices across two periods, calculates percentage fluctuations, flags changes above the threshold, and identifies the largest change.

CASE STUDY 10: ATM Transaction Sequence Simulation

1. Problem Statement

Write a Python program to simulate an ATM transaction sequence. It must process deposits and withdrawals, update the balance, reject invalid transactions, and display a transaction summary.

2. Objectives

- Use variables to maintain an account balance.
- Use conditions to validate transactions.
- Use a loop to process several transactions.
- Count successful deposits, withdrawals, and rejected transactions.

3. Assumptions

109. The opening balance must not be negative.
110. The number of transactions must not be negative.
111. Each transaction amount must be greater than zero.
112. Withdrawals cannot exceed the available balance.
113. The program accepts D for deposit and W for withdrawal.

4. Algorithm

114. Start.
115. Input opening balance and transaction count.
116. Validate the opening balance and count.
117. Initialize counters for deposits, withdrawals, and rejected transactions.
118. For each transaction, input type and amount.
119. Reject non-positive amounts.
120. For a deposit, add amount to balance and increase deposit counter.
121. For a withdrawal, check available balance; update balance if sufficient, otherwise reject.
122. Reject unknown transaction types.
123. Display final balance and counters.
124. Stop.

5. Flowchart

Draw this flow in Word using Insert → Shapes. Use ovals for Start/Stop, parallelograms for input/output, rectangles for processing, diamonds for decisions, and arrows for flow.

6. Pseudocode

```
START
INPUT balance, n
IF balance < 0 OR n < 0 THEN DISPLAY invalid input
ELSE
    deposits = 0; withdrawals = 0; rejected = 0
    REPEAT n times
        INPUT transaction_type, amount
        IF amount <= 0 THEN rejected = rejected + 1
        ELSE IF type = "D" THEN balance = balance + amount; deposits = deposits + 1
        ELSE IF type = "W" AND amount <= balance THEN balance = balance - amount; withdrawals =
```

```
withdrawals + 1
        ELSE rejected = rejected + 1
    END REPEAT
    DISPLAY balance and counters
END IF
STOP
```

7. Python Source Code

```
balance = float(input("Enter opening balance: "))
n = int(input("Enter number of transactions: "))

deposits = 0
withdrawals = 0
rejected = 0

if balance < 0 or n < 0:
    print("Invalid opening balance or transaction count.")
else:
    for i in range(n):
        print("\nTransaction", i + 1)
        transaction_type = input("Enter D for deposit or W for withdrawal: ").upper()
        amount = float(input("Enter amount: "))

        if amount <= 0:
            print("Invalid amount.")
            rejected += 1
        elif transaction_type == "D":
            balance += amount
            deposits += 1
            print("Deposit successful.")
        elif transaction_type == "W":
            if amount <= balance:
                balance -= amount
                withdrawals += 1
                print("Withdrawal successful.")
            else:
                print("Insufficient balance. Transaction rejected.")
                rejected += 1
        else:
            print("Invalid transaction type.")
            rejected += 1

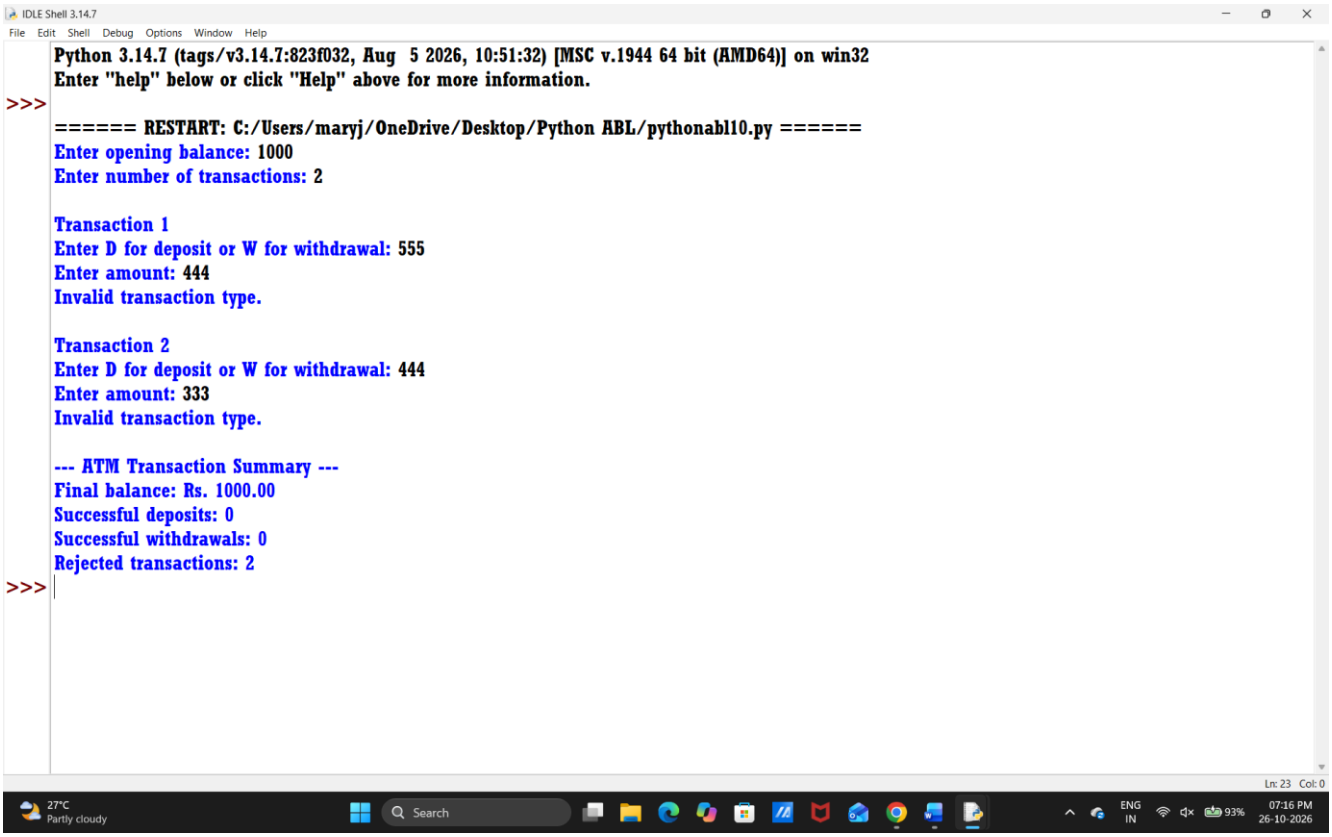
    print("\n--- ATM Transaction Summary ---")
    print(f"Final balance: Rs. {balance:.2f}")
    print("Successful deposits:", deposits)
    print("Successful withdrawals:", withdrawals)
    print("Rejected transactions:", rejected)
```

8. Test Cases and Output Screenshots

Input / setup: Opening 1000; D 500; W 300; W 1500

Expected output: Final balance ₹1200.00; deposits 1; withdrawals 1; rejected 1

Test note: Three transactions: deposit 500, withdraw 300, attempt withdraw 1500



9. Conclusion

The program simulates a sequence of ATM transactions using loops, variables, and conditional statements. It updates the account balance only for valid transactions and summarizes the results.

Final Overall Conclusion

These ten case studies apply the Python concepts covered in the assigned syllabus, including variables, data types, operators, conditional statements, loops, strings, lists, nested lists, tuples, and tuple unpacking. Each program should be run and tested with valid and invalid inputs. The student should insert genuine output screenshots, review the flowcharts, and verify all calculations before submitting the consolidated report.

Submission Checklist

- Fill in your name, roll number, branch, and section.
- Run all 10 programs and correct any errors.
- Insert actual output screenshots for every case study.
- Draw or recreate each flowchart neatly.
- Check the test outputs and formatting.
- Export this document to PDF.
- Upload the PDF and Python source files to the required GitHub repository named Python-Activity-Based-Learning.